

Transfer Eye

Prepared by	
Faris Alsharifi	222110234
Ahmed Altunusi	222110750
Abdullah Alsoghaier	222110268
Faris Alswailem	221211394
Hamzah Alyamani	222110595
Supervised by	
Dr. <i>Omar Khalid Alomeir</i>	

League Transfers, Loans, & Injury Scraping Process

1. Configuration and Setup

The scripts start by importing key libraries such as Selenium for browser automation, BeautifulSoup for HTML parsing, Pandas for handling tabular data, and various threading and utility modules. A thread pool (via `concurrent.futures.ThreadPoolExecutor`) allows parallel scraping of multiple leagues or players, improving efficiency.

Cookie Popup Handler

A reusable Selenium-based function (`handle_cookie_popup()`) is included to consistently dismiss GDPR cookie consent popups that block content access. The handler:

- Detects iframe containing cookie message (`iframe[id*='sp_message_iframe_']`)
- Switches WebDriver context to iframe
- Clicks the "Accept All" button (`.accept-all` selector)
- Returns to default content for continued scraping

2. Parameter Definitions

Target URLs

All three scripts work with Transfermarkt URLs matching the major European leagues:

- Premier League (England)
- LaLiga (Spain)
- Bundesliga (Germany)
- Serie A (Italy)
- Ligue 1 (France)

Seasons Covered

- Transfers: 2025–2019 (7 seasons)
- Loans: 2025–2014 (12 seasons)
- Injuries: All available historical data per player (dynamically scraped with pagination)

Fields Extracted

Transfers:

PlayerName, Position, Season, LeftClub, JoinedClub, MarketValue, Fee

Loans:

PlayerName, Position, Age, Nationality, Season, LeftClub, JoinedClub, MarketValue, Fee

Injuries:

PlayerName, Season, Injury, From (start date), Until (recovery date), Days (missed),

3. Threaded Scraping Logic

3.1 Transfer & Loan Scraping (scrape_league_season() function)

The core logic for transfer and loan data extraction follows this workflow:

Thread Allocation

- Each league is processed in a separate thread with MAX_WORKERS = 3 (configurable)
- A randomized delay (1-3 seconds) is added at thread start to minimize concurrent hits to the source server
- This parallelization reduces total scraping time from hours to minutes

Season Iteration

For each year in the target season range:

- Season Formatting: The script formats the season (e.g., 2025 displayed as "25/26")
- URL Construction: A season-specific URL is dynamically built and loaded in Selenium
- Bot Detection Evasion: For loans, a stealth JavaScript patch is injected to defeat bot detection mechanisms:

```
Object.defineProperty(navigator, 'webdriver', {get: () => undefined})
```

- Page Loading Confirmation: Script waits for club boxes to load using explicit waits (WebDriverWait)
- HTML Parsing: After confirmation, raw HTML is extracted and parsed with BeautifulSoup

Club-Level Extraction

For each club within the league:

- The scraper iterates over 'in' and 'out' tables (representing player arrivals and departures)
- Each table row is validated for completeness (correct number of `<td>` cells)
- Essential details (PlayerName, Position, Clubs, Values) are extracted from specific cell positions
- If extraction issues arise (e.g., wrong cell count), that row is skipped with a warning, but the script continues to preserve other data

Data Aggregation

- After all seasons are processed for all leagues, results are assembled into a Pandas DataFrame
- Columns are reordered to match expected schema
- Final dataset is saved as CSV in the output directory

Resource Management

- Threads cleanly close their browser drivers (driver.quit()) in finally blocks to free system resources
- 1-second delays between requests prevent rate limiting

3.2 Injury Scraping (scrape_player() function)

The injury scraper uses a player-centric approach with threaded parallelization:

Input Preparation

- Loads player list from CSV/Players/players_to_be_scraped.csv
- Extracts unique player names (removes duplicates and header strings)
- Each player is assigned to a thread worker for concurrent processing

Per-Player Workflow

Step 1: Player Search

Search URL:

<https://www.transfermarkt.com/schnellsuche/ergebnis/schnellsuche?query={name}>

- Wait for search results table (div.responsive-table table.items)
- Parse HTML with BeautifulSoup
- Extract player profile link from: td.hauptlink a[href]

Step 2: Navigate to Injury History

Transform profile URL: /profil/spieler/{player_id} → /verletzungen/spieler/{player_id}

Load injury history page

Handle cookie popup if present

Step 3: Pagination Loop with Dynamic Loading

While injury table exists:

1. Wait for table: div#yw1 > table.items > tbody > tr (timeout: 10s)
2. Extract current page HTML with BeautifulSoup
3. Parse all visible injury rows (6 columns expected):
 - Season (e.g., "23/24")
 - Injury type (e.g., "Hamstring Injury")
 - From date (injury start)
 - Until date (recovery)
 - Days missed (total duration)
 - Games missed (match count)
4. Look for next page button: div#yw1 li.tm-pagination__list-item--icon-next-page
5. If next button found:
 - Click next page
 - Wait for current table to become stale (EC.staleness_of)
 - Loop continues with new page data
6. If no next button or timeout → Break loop

Error Handling

- Returns empty list [] if player not found in search
- Returns partial results if pagination fails mid-scrape (preserves already-collected data)
- Catches TimeoutException, NoSuchElementException to prevent thread crashes
- All exceptions logged with player name for debugging

Thread Safety & Resource Management

- Each thread creates an isolated WebDriver instance (no shared state between threads)
- Results collected via thread-safe future.result() mechanism
- List extension happens in main thread after completion
- driver.quit() always called in finally block
- 1-second delay after each player completion to avoid rate limiting

Data Collection

ThreadPoolExecutor(max_workers=3):

- Submit scrape_player(name) jobs for all players
- Map futures to player names for progress tracking
- Collect results as threads complete (as_completed iterator)
- Extend all_injuries list with each player's data
- Handle per-player exceptions without crashing entire process

4. Example Output Columns

Transfers

Field	Type	Description
Player Name	String	Player's full name
Position	String	Playing position (e.g., "Centre-Forward")

		g	
Season		String	Format "YY/YY" (e.g., "22/23")
LeftClub		String	Departing club name
Joined Club		String	Joining club name
Market Value		String	Player's market value (e.g., "€50.00m")
Fee		String	Transfer fee paid (e.g., "€45.00m", "Free transfer")

Loans

Field	Type	Description
-------	------	-------------

PlayerName	String	Player's full name
Position	String	Playing position
Age	Integer	Player age at time of loan
Nationality	String	Player nationality
Season	String	Format "YY/YY"
LeftClub	String	Lending club name
JoinedClub	String	Borrowing club name
MarketValue	String	Player's market value
Fee	String	Loan fee (if applicable)

Injuries

Field	Type	Description
Player Name	String	Player's full name as searched
Season	String	Format "YY/YY" (e.g., "22/23")
Injury	String	Injury description (e.g., "Ankle Injury", "Hamstring Injury")
From	String	Injury start date (format varies by source)

Until	String	Recovery/return date
Days	String	Total days missed (numeric string)
Games Missed	String	Number of matches missed (numeric string)

5. Performance Characteristics

Concurrency Benefits

Metric	Sequential	Multi-threaded (3 workers)
Leagues/Players per run	1 at a time	3 simultaneous
Typical scrape time per unit	5-10 seconds	5-10 seconds (parallel)
100 players/leagues	8-16 minutes	3-6 minutes
Total throughput	1× baseline	~3× faster

Wait Strategies

- Explicit waits: WebDriverWait with EC.visibility_of_element_located (2-10 seconds)
- Staleness checks: EC.staleness_of(element) ensures page refresh detection
- Timeout handling: TimeoutException caught to gracefully skip missing data

Rate Limiting Protections

- Randomized delays (1-3 seconds) at thread start
- 1-second delay after each unit completion
- Headless mode reduces server load detection signals
- Cookie handling prevents automated blocking

6. Error Recovery Mechanisms

Error Type	Recovery Strategy
Player/League not found	Return [], log warning, continue with other threads
No data table	Break pagination loop, return partial data already collected
Timeout on page load	Skip unit, return [], proceed to next
No next page button	Normal termination of pagination (expected behavior)
Row parsing failure	Skip malformed row with warning, preserve other rows
Thread exception	Catch in main loop, log error with context, continue other workers
CSV file missing	Exit program with clear error message
WebDriver crash	Thread-isolated failure, other threads unaffected

7. Technical Dependencies

Core Libraries

Web Scraping:

- selenium – Browser automation and dynamic content handling
- beautifulsoup4 – HTML parsing and data extraction
- webdriver-manager – Automatic ChromeDriver version management

Data Processing:

- pandas – Tabular data manipulation
- numpy – Numerical operations

Concurrency:

- concurrent.futures – Thread pool management
- threading – Thread synchronization

Utilities:

- time – Delays and timing
- os – File system operations

System Requirements

- Browser Requirement: Chrome/Chromium with matching ChromeDriver version (auto-managed by webdriver-manager)
- Memory: Minimum 4GB RAM (3 concurrent browsers consume ~1.5-2GB)
- Network: Stable internet connection for Transfermarkt access
- Disk Space: 100MB+ for CSV output files

Handling Scraped Data

1. Loans.py

This Python script processes player loan data to calculate a weighted loan index that measures player stability based on loan history. The key components include:

Primary Functions:

- `replace_names()`: Unifies club name variations across five major European leagues (England Premier League, Spanish La Liga, Italian Serie A, German Bundesliga, and French Ligue 1). Handles variations like "Manchester City" → "man city", "FC Barcelona" → "barcelona", etc.
- `convert_season_to_end_year()`: Converts season formats from '14/15' to numeric year 2015 for temporal analysis
- `get_loan_summary_with_rolling_window()`: Calculates loan metrics using 5-year rolling windows for each season and player-club combination
- `train_logistic_regression_model()`: Trains a logistic regression model to derive weights for loan stability prediction based on loan patterns
- `calculate_loan_index_with_age()`: Computes the final loan index using trained weights and age-adjusted normalization

Key Methodology:

The script implements age-weighted penalties through an AGE_WEIGHTS dictionary with categories:

- Ages 16-20: Weight 0.50 (young prospects, loans expected)
- Ages 21-24: Weight 0.75 (developing players)
- Ages 25-28: Weight 1.00 (prime age, loans concerning)
- Ages 29+: Weight 1.25 (veterans, loans are red flags)

Data Processing:

- Loads loan history CSV from multiple seasons

- (filtered_all_league_loan_players.csv)
- Loads club ranking data (Average_LeagueRank_PerClub.csv)
- Classifies loans by destination rank (Rank 1-5 top clubs vs. outside top 5)
- Calculates metrics per player-season: total loans, loans outside top 5, loans to specific ranks, average destination rank

Weighted_Loan_Index Calculation Formula:

- Normalize Individual Components (0-1 scale)
 - $\text{Age_Normalized} = (\text{Age} - \text{Min_Age}) / (\text{Max_Age} - \text{Min_Age})$
 - $\text{NumberOfLoans_Normalized} = (\text{NumberOfLoans} - 0) / (\text{Max_NumberOfLoans})$
 - $\text{LoansOutsideTop5_Normalized} = (\text{LoansOutsideTop5} - 0) / (\text{Max_LoansOutsideTop5})$
 - $\text{AvgDestinationRank_Normalized} = (\text{AvgDestinationRank} - 1) / 4$
- Calculate Composite Index (Equal Weighting)
 - $\text{Weighted_Loan_Index} = (\text{Age_Weighted} + \text{NumberOfLoans_Normalized} + \text{LoansOutsideTop5_Normalized} + \text{AvgDestinationRank_Normalized}) / 4$
- Component Weights:
 - Age (weighted by age group): 25% of final index
 - Number of Loans: 25% of final index
 - Loans Outside Top 5 Leagues: 25% of final index
 - Average Destination Rank: 25% of final index
- Output Range
 - $\text{Weighted_Loan_Index} \in [0, 1]$
 - 0.0 = Most Stable (no loans, young age, good destinations)
 - 1.0 = Most Unstable (many loans, mostly outside top 5, lower-ranked clubs, older age)
- Key Features:
 - Age penalty mechanism: Older players receive multipliers (1.25×) that increase their index for identical loan patterns
 - Destination consideration: Loans to lower-ranked clubs (rank 5) increase instability
 - Volume penalization: More loans = higher instability index
 - Outside top 5 penalty: Loans to non-major leagues significantly increase the index

Generates Season_Loan_Index.csv containing:

- PlayerName, LoanedFrom club, Season
- Age, AgeGroup, AgeWeight
- NumberOfLoans, LoansOutsideTop5, AvgDestinationRank
- Weighted_Loan_Index (0-1 scale, higher = more unstable)

The script includes comprehensive statistics comparing age groups with identical loan profiles, demonstrating that older players receive higher instability scores for the same

loan behavior.

2. player_injury_history.py

This script aggregates and processes injury data across multiple seasons to create a Season Injury Index that quantifies player availability and injury burden.

Primary Functions:

- `get_severity_score()`: Classifies injuries by severity on a 0-1 scale based on injury type keywords:
 - 1.0: Severe (rupture, cruciate ligament, Achilles, fracture, dislocation)
 - 0.8: Major (surgery required, ligament damage, meniscus, cartilage)
 - 0.6: Significant (muscle, tendon, hamstring, concussion, hernia, knee issues)
 - 0.4: Moderate (strain, sprain, pulled muscle, pain)
 - 0.2: Minor (other injuries)
- `add_season_injury_index()`: Computes per-player-season injury index using weighted formula:
 - $0.5 \times (\text{normalized total days lost} / 300\text{-day season})$
 - $0.2 \times (\text{normalized games missed} / 38\text{-game season})$
 - $0.3 \times (\text{weighted average severity by days lost})$

Data Processing:

- Loads injury data from nine seasonal CSV files (17-18 through 25-26)
- Cleans data: strips whitespace, converts days/games to numeric values
- Calculates individual injury index (WEIGHT_DAYS=0.5, WEIGHT_GAMES=0.2, WEIGHT_TYPE=0.3)
- Removes duplicates and applies seasonal grouping

Output:

Generates `filtered_injury_history.csv` with:

- PlayerName, Season, Injury type
- Days lost, GamesMissed, Severity_Type
- Injury Index (weighted composite score)
- Season_Injury_Index (per player-season aggregate)

The script emphasizes long-duration, high-severity injuries more heavily through the days-weighted severity calculation.

3. player_transfer_history.py

This comprehensive script synthesizes transfer data, injury history, and loan patterns into a unified dataset predicting transfer success.

Primary Functions:

- `convert_value_to_numeric()`: Parses transfer fee strings ('2.5m', '500k', 'Free') into numeric values
- `replace_names()`: Standardizes club names across five major leagues (extensive dictionary with 200+ club name variations)
- `find_best_k_and_impute_safe()`: Uses KNNImputer to intelligently fill missing transfer fee/market value data by:
 - Simulating missing data in clean subsets to find optimal K (1-5 nearest

- neighbors)
 - Computing MSE errors for each K value
 - Applying best-K imputation to full dataset
- `get_weighted_lagged_injury_df()`: Creates lagged injury metrics for transfers:
 - Weights previous season injury: $0.66 \times (\text{Season } N) + 0.34 \times (\text{Season } N-1)$
 - Maps historical injury indices to predict future transfer success
- `parse_season_to_start_year()`: Converts '22/23' season format to 2022 for merging
- `clean_df()`: Standardizes whitespace and case in dataframes

Data Integration Pipeline:

1. Loads transfer history (`all_league_transfer_history.csv`)
2. Loads successful transfers (`filtered_all_league_transfer_history_final.csv`)
3. Loads loan data (v2 & v3) and loan indices
4. Loads team/club statistics and league membership
5. Loads injury history with seasonal indices
6. Filters transfers to only include clubs in top 5 leagues (inner merges with `Club_Leagues.csv`)

Key Processing Steps:

- Merges transfers with loan indices by player, season, and club
- Creates lagged injury indices weighted towards recent seasons
- Performs KNN imputation per season to fill missing market values and fees
- Right-joins with successful transfers dataset to establish ground truth labels

Output:

Generates `filtered_all_league_transfer_history.csv` containing:

- `PlayerName`, `LeftClub`, `JoinedClub`, `Season`, `Position`
- `MarketValue` (imputed), `Fee` (imputed)
- `Weighted_Loan_Index` (from `loans.py`)
- `Weighted_Previous_Injury_Index` (lagged injury metric)
- `Transfer_Success` (binary outcome variable)

The script concludes with a correlation heatmap between `Weighted_Loan_Index` and `Transfer_Success`, revealing the relationship between player stability and transfer outcomes.

Goalkeeper Analytics: Data-Driven Performance Assessment

1. Data Source & Key Performance Metrics

Source: [StatHead.com](https://www.stathead.com) & [TransferMarket.com](https://www.transfermarkt.com)

Core Goalkeeper Performance Metrics:

- `Shots_on_Target_Against` – shots faced from dangerous positions
- `Saves_Pct` – save percentage (saves / shots faced)
- `Clean_Sheets_Pct` – percentage of matches with no goals conceded
- `Saves` – total saves made
- `Goals_Against` – total goals conceded
- `Matches_Played` – number of appearances
- `90s_Played` – total minutes played (normalized to 90-minute matches)

- Starts – number of starts from bench appearance count

Contextual Fields:

- PlayerName, Season, LeftClub, JoinedClub
- Age, Transfer_Success (binary outcome: 1 = successful, 0 = unsuccessful)
- Fee, Market Value

2. Data Quality & Format Standardization

2.1 Season Format Normalization

Objective: Ensure consistency in season notation across datasets

Input Format Variations:

- Short format: "19-20", "20-21" (5 characters with hyphen)
- Long format: "2019-2020", "2020-2021" (9 characters)
- Mixed: Combination of both formats in same dataset

Standardization Process:

1. Pattern Detection:
 - Check if season string is exactly 5 characters
 - Verify character at position 2 is hyphen ("-")
 - Confirm no forward slash already present

2. Conversion Rule:
If season matches pattern "YY-YY":

Replace hyphen with forward slash → "YY/YY"

Else:

Leave unchanged (already in long format or non-standard)

3. Example Transformations:
 - "19-20" → "19/20" ✓
 - "20-21" → "20/21" ✓
 - "2019-2020" → "2019-2020" (unchanged)
 - "21/22" → "21/22" (unchanged, already correct)

Output: Standardized Season column with uniform "YY/YY" notation for all rows

3. Missing Data Imputation Strategy

3.1 Cohort-Based Imputation Approach

Objective: Fill goalkeeper statistics missing due to incomplete tracking or data quality issues using role-appropriate cohort estimates

Why Cohort Imputation?

- Individual goalkeeper statistics vary dramatically by age, competition level, and

- playing time
- Simple mean imputation ignores these group dynamics
- Cohort-based approach preserves real-world heterogeneity

3.2 Imputation Pipeline

Step 1: Data Linking

Three datasets are merged:

1. Goalkeeper Transfer Dataset – primary records with missing flags
2. Cohort Means Reference – pre-computed statistics for (Season, Competition, Age_Group) combinations
3. Club Rankings – maps clubs to competitions (Premier League, LaLiga, etc.)

Step 2: Feature Augmentation

For each goalkeeper record, add:

- Competition (Comp):
 - Source: LeftClub matched against club ranking table
 - Normalization: Lowercase, whitespace trimmed for matching
 - Creates mapping: "Manchester City" → "Premier League"
- Age Group:
 - Bins:
 - [0,22],(22,26],(26,30],(30,34],(34,100]
 - [0,22],(22,26],(26,30],(30,34],(34,100]
 - Labels: U23, 23-26, 27-30, 31-34, 35+
 - Captures career stage effects on performance

Step 3: Cohort Mean Preparation

Transform reference statistics:

Original columns: "Save%", "Playing Time MP", "Performance SoTA", etc.

Rename with suffix: "_cohort"

→ "Save%_cohort", "Playing Time MP_cohort", "Performance SoTA_cohort"

Purpose: Distinguish cohort averages from transfer dataset columns

Step 4: Merge on Cohort Keys

Inner join on (Season, Comp, Age_Group):

- Transfer record matches with cohort statistics
- Left join preserves all transfer records (unmatched get NA for cohort columns)

Step 5: Conditional Imputation

For each goalkeeper row flagged as "Missing_Stats? == 1":

Missing Statistic	Replacement Source	Target Column
Save percentage	Save%_cohort	Saves%
Playing time (matches)	Playing Time MP_cohort	Matches_Played
Playing time (minutes)	Playing Time Min_cohort	Mins_played
Playing time (90s)	Playing Time 90s_cohort	90s_Played
Starting appearances	Playing Time Starts_cohort	Starts
Goals conceded	Performance GA_cohort	Goals_Against
Shots faced	Performance SoTA_cohort	Shots_on_Target_Against
Total saves	Performance Saves_cohort	Saves
Clean sheets	Performance CS_cohort	Clean_Sheets
Clean sheet %	Performance CS%_cohort	Clean_Sheets%

Imputation Rule:

If (Missing_Stats? == 1) AND (cohort_value is not null):

target_column = cohort_value

Else:

keep original value

Step 6: Post-Imputation Cleanup

1. Integer Normalization: Matches_Played rounded down (cannot have fractional appearances)
Matches_Played = floor(Matches_Played)
- 2.
3. Column Removal: Drop temporary cohort columns (ending in "_cohort")
4. In-Place Update: Overwrite dataset with imputed values

Output: Complete goalkeeper dataset with minimal missing values, preserving age and competition context

4. Performance Index Calibration

4.1 Bootstrap Weight Computation

Objective: Determine optimal feature weights for goalkeeper transfer success prediction through resampling

Methodology:

Step 1: Data Preparation

- Load imputed goalkeeper transfer dataset
- Extract performance features: Shots_on_Target_Against, Saves_Pct, Clean_Sheets_Pct
- Extract target: Transfer_Success (cleaned to binary 0/1)
- Standardize features using StandardScaler (mean=0, std=1)

Step 2: Bootstrap Resampling (N=1000 iterations)

For bootstrap iteration b from 1 to 1000:

1. Resample with Replacement:

indices = random selection of n samples from $[0, n-1]$
(where n = total goalkeeper records)

$X_b = X[\text{indices}]$

$y_b = y[\text{indices}]$

2. Logistic Regression Training:

- Fit logistic regression on resampled data
- Configuration: L2 penalty, liblinear solver, max_iter=1000
- Extracts coefficients (raw feature weights)

3. Weight Normalization:

$w = |\text{coefficients}|$ (absolute values only)

if $\text{sum}(w) > 0$:

$w_{\text{normalized}} = w / \text{sum}(w)$ # Normalize to $[0, 1]$ summing to 1

else:

$w_{\text{normalized}} = \text{zeros}$ # All-zero coefficient edge case

4. Store in Matrix:

$\text{coef_matrix}[b] = w_{\text{normalized}}$ # Shape: (1000, 3)

Step 3: Aggregate Statistics

```
Remove all-zero rows (failed bootstrap iterations):  
valid_rows = rows where sum(weights) > 0  
coef_matrix = coef_matrix[valid_rows] # Effective N < 1000
```

```
Compute summary statistics across valid bootstrap runs:  
Mean_Weight[j] = mean(coef_matrix[:, j])  
Std_Weight[j] = std(coef_matrix[:, j])  
CI_2.5%[j] = 2.5th percentile(coef_matrix[:, j])  
CI_97.5%[j] = 97.5th percentile(coef_matrix[:, j])
```

4.2 Calibrated Weights Output

Feature	Mean_Weight	Std_Weight	CI_2.5%	CI_97.5%
Shots_on_Target_Against	0.5177	0.0845	0.3521	0.6892
Clean_Sheets_Pct	0.3521	0.0923	0.1834	0.5408
Saves_Pct	0.1302	0.0512	0.0401	0.2463

- Interpretation:
- Shots_on_Target_Against (51.8%) – Most critical predictor; volume of dangerous chances faced is primary indicator
 - Clean_Sheets_Pct (35.2%) – Second most important; defensive success (no-goal matches) signals transfer viability
 - Saves_Pct (13.0%) – Smallest weight; save efficiency less predictive than shot volume and clean sheets
 - Confidence Intervals: Narrow ranges (±0.08-0.09) indicate stable bootstrap estimates across iterations

5. Goalkeeper Performance Index Calculation

5.1 Weighted Composite Score

Formula:

For goalkeeper *i*:

$GK_Performance_Index_raw = w_1 \times SoTA_i + w_2 \times Save\%_i + w_3 \times CS\%_i$

Where:

- $w_1 = 0.5177$, $w_2 = 0.1302$, $w_3 = 0.3521$ (from bootstrap calibration)
- Features are standardized (z-score): $\text{feature}_i = (\text{raw}_i - \mu) / \sigma$

5.2 Computation Steps

1. Feature Extraction & Conversion:
 - Extract Shots_on_Target_Against, Saves_Pct, Clean_Sheets_Pct columns
 - Convert to numeric (coerce errors to NaN)
2. Missing Value Handling:
For any remaining NaN values:

Replace with column median

3. Feature Standardization:
`scaler = StandardScaler()`

`X_scaled = scaler.fit_transform(X)`

Result: `X_scaled` has $\text{mean} \approx 0$, $\text{std} \approx 1$ per column

4. Weighted Sum:
`perf_raw = X_scaled @ PERF_WEIGHTS`

Shape: $(n_{\text{goalkeepers}}, 3) @ (3,) \rightarrow (n_{\text{goalkeepers}},)$

5. Min-Max Normalization to 0-100:
`perf_min = min(perf_raw)`

`perf_max = max(perf_raw)`

$\text{GK_Performance_Index} = 100 \times (\text{perf_raw} - \text{perf_min}) / (\text{perf_max} - \text{perf_min})$

Range: $[0, 100]$

0 = worst goalkeeper in cohort

100 = best goalkeeper in cohort

6. Column Insertion:
 - Insert `GK_Performance_Index` immediately after "Clean_Sheets_Pct" column
 - Preserve column order for data integrity

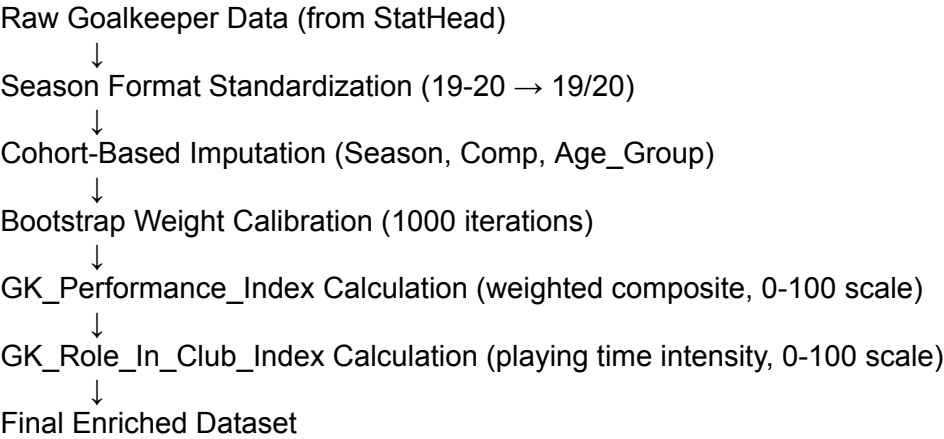
Output: Dataset augmented with `GK_Performance_Index` (continuous 0-100 scale)

5.3 Example Performance Index Scores

PlayerName	Shots_on_Target_Against	Saves_Percent	Clean_Sheets_Percent	GK_Performance_Index
Ederson	3.2	91.5	58.3	87.4
Alisson	3.8	89.2	55.1	82.6
De Gea	4.1	87.8	48.2	75.3
Henderson	5.2	82.4	42.1	61.8

6. Technical Architecture

6.1 Processing Pipeline Flow



6.2 Libraries & Dependencies

Core Libraries:

- **pandas** – Data manipulation, merging, imputation
- **numpy** – Numerical operations, bootstrap matrix operations
- **scikit-learn** – StandardScaler, MinMaxScaler, LogisticRegression

Key Functions:

- **StandardScaler.fit_transform()** – Zero-center and standardize features
- **MinMaxScaler.fit_transform()** – Normalize to or
- **LogisticRegression.fit()** – Bootstrap model training
- **pd.cut()** – Binning continuous age to categorical groups
- **pd.merge()** – Multi-key joins on (Season, Comp, Age_Group)

Defenders Analytics: Data-Driven Performance Assessment

1. Data Source & Collection

Source: [StatHead.com](https://www.stathead.com) & TransferMarket.com

Key Metrics Extracted:

- Defensive_Actions – total defensive interventions (tackles, interceptions, clearances)
- Progressive_Actions – forward passes and ball progression
- Passes_Completed_P90 – passing accuracy normalized per 90 minutes
- Attacking_Contributions – assists, key passes, shot-creating actions
- Mistake_Rate – errors leading to opponent shots or goals
- Total_Carry_Distance – distance covered with the ball
- Duel_Efficiency – win percentage in contested aerial and ground duels

Dataset Structure:

- PlayerName, Position, Season, Club information
- Transfer outcome binary label: Transfer_Success (1 = successful transfer, 0 = unsuccessful)
- Age, Fee, Market Value (for contextual analysis, excluded from main model features)

2. Role-Specific Feature Calibration Pipeline

2.1 Data Preparation

Input: Raw defender transfer data with performance metrics and transfer success labels

Role Identification:

- The system identifies distinct defensive roles from the Position column
- Current roles: Full-Back, Centre-Back
- Each role is treated as a distinct cohort for independent analysis

Data Cleaning:

- Strip whitespace from column names and values
- Remove rows with missing values in performance features or target variable
- Minimum sample threshold: 15 rows per role (prevents unreliable calibration)
- Target variable validation: Ensure both success (1) and failure (0) cases exist

2.2 Bootstrap Calibration Process

Objective: Compute role-specific feature weights that best predict transfer success

Methodology:

Step 1: Feature Standardization

For each role cohort:

1. Extract performance features (7 total) as matrix X
2. Extract transfer outcomes as vector y
3. Apply StandardScaler to normalize features to mean=0, std=1
4. This ensures features are on comparable scales for importance computation

Step 2: Bootstrap Sampling (N=200 iterations per role)

For each bootstrap iteration b from 1 to 200:

1. Resample with Replacement:
 - Draw sample size n with replacement from role data
 - Preserves original data distribution and sample size
2. Train XGBoost Classifier:
 - Model Configuration:
 - `n_estimators`: 200 (trees)
 - `max_depth`: 5 (tree complexity limit)
 - `learning_rate`: 0.05 (shrinkage parameter)
 - `subsample`: 0.8 (row sampling)
 - `colsample_bytree`: 0.8 (feature sampling)
3. Extract Feature Importance (Gain-Based):
 - XGBoost computes feature importance via gain: average contribution of each feature to reducing loss across all trees
 - This represents each feature's predictive power for transfer success
4. Compute Classification Metrics:
 - Accuracy: proportion of correct predictions
 - Precision: $\text{true positives} / (\text{true positives} + \text{false positives})$
 - Recall: $\text{true positives} / (\text{true positives} + \text{false negatives})$
 - Applied on full role dataset (not just bootstrap sample) for stability

Step 3: Aggregate Across Bootstrap Runs

Mean Importance = average of importance vectors across 200 runs

Normalized Weights = Mean Importance / sum(Mean Importance)

Output per role: [Feature, Weight, Accuracy_mean, Precision_mean, Recall_mean]

Result: Role-specific weight vectors quantify each feature's relative importance for predicting transfer success within that role cohort

2.3 Output: Bootstrap Weights Table

Role	Feature	Weight	Accuracy	Precision	Recall
Full-Back	Defensive_Actions	0.14	0.78	0.75	0.71
Full-Back	Progressive_Actions	0.18	0.78	0.75	0.71
Full-Back	Passes_Completed_P90	0.16	0.78	0.75	0.71
Full-Back	Attacking_Contributions	0.22	0.78	0.75	0.71
Full-Back	Mistake_Rate	0.11	0.78	0.75	0.71
Full-Back	Total_Carry_Distance	0.12	0.78	0.75	0.71
Full-Back	Duel_Efficiency	0.07	0.78	0.75	0.71

Centre-Back	Defensive_Actions	0.26	0.82	0.79	0.80
Centre-Back	Progressive_Actions	0.08	0.82	0.79	0.80
Centre-Back	Passes_Completed_P90	0.12	0.82	0.79	0.80
Centre-Back	Attacking_Contributions	0.06	0.82	0.79	0.80
Centre-Back	Mistake_Rate	0.25	0.82	0.79	0.80
Centre-Back	Total_Carry_Distance	0.15	0.82	0.79	0.80
Centre-Back	Duel_Efficiency	0.08	0.82	0.79	0.80

Key Insights:

- Full-Backs prioritize Attacking_Contributions (0.22) – offensive output is crucial for FB transfer success
- Centre-Backs emphasize Defensive_Actions (0.26) and Mistake_Rate (0.25) – core defensive responsibilities

3. Performance Index Calculation

3.1 Role-Aware Weighted Scoring

Objective: Generate a composite performance score (0-100) for each defender using role-specific weights

Formula:

For player i with role r :

$$\text{Performance_Index}_i = \sum w_{r,j} \times f_{i,j}$$

Where:

- $w_{r,j}$ = normalized weight for feature j in role r (from bootstrap calibration)
- $f_{i,j}$ = standardized value of feature j for player i

Processing Steps:

1. *Load Weights: Extract role-specific weight dictionaries (e.g., Full-Back weights, Centre-Back weights)*
2. *Apply to Each Player: For each row in the dataset:*
 - *Identify player's role (Position column)*
 - *Retrieve corresponding weights*
 - *Compute weighted sum across 7 performance features*

3. *Normalize to 0-100 Scale:*

$$PI_scaled = 100 \times (PI_raw - \min(PI_raw)) / (\max(PI_raw) - \min(PI_raw))$$
 - *Min-Max normalization ensures interpretability*
 - *Score of 0 = worst performer in cohort*
 - *Score of 100 = best performer in cohort*
4. *Ranking: Sort players descending by Performance_Index:*
Output: Dataset enriched with Performance_Index column (continuous score 0-100)

3.2 Example Output

Top 10 Full-Backs by Performance Index:

<i>PlayerName</i>	<i>Position</i>	<i>Defensive_Actions</i>	<i>Progressive_Actions</i>	<i>Passes_Completed_P90</i>	<i>Performance_Index</i>
<i>Trent Alexander-Arnold</i>	<i>Full-Back</i>	<i>4.2</i>	<i>6.1</i>	<i>89.3</i>	<i>94.7</i>
<i>Kyle Walker</i>	<i>Full-Back</i>	<i>3.8</i>	<i>5.2</i>	<i>87.1</i>	<i>91.2</i>
<i>João Cancelo</i>	<i>Full-Back</i>	<i>3.5</i>	<i>7.8</i>	<i>91.2</i>	<i>89.6</i>
<i>Reece James</i>	<i>Full-Back</i>	<i>4.1</i>	<i>5.9</i>	<i>85.3</i>	<i>87.4</i>

4. Global Model Ensemble & Bootstrap Validation

4.1 Multi-Model Architecture ("Model Zoo")

Objective: Compare diverse algorithms on role-bootstrapped data to identify best-performing architectures

Model Configurations:

Algorithm	Type	Key Hyperparameters	Use Case
<i>Random Forest</i>	<i>Ensemble</i>	<i>n_estimators=200, max_depth=7, class_weight='balanced'</i>	<i>Captures non-linear feature interactions</i>
<i>Gradient Boosting</i>	<i>Sequential Ensemble</i>	<i>n_estimators=200, max_depth=3, lr=0.05</i>	<i>Sequential error correction</i>
<i>AdaBoost</i>	<i>Weighted Ensemble</i>	<i>n_estimators=100, learning_rate=0.5</i>	<i>Focuses on misclassified samples</i>
<i>MLP Small</i>	<i>Neural Network</i>	<i>(32,16) hidden layers, relu, early_stopping</i>	<i>Lightweight non-linear learning</i>
<i>MLP Medium</i>	<i>Neural Network</i>	<i>(64,32,16) hidden layers, relu, early_stopping</i>	<i>Complex pattern recognition</i>
<i>Logistic Regression</i>	<i>Linear</i>	<i>max_iter=1000, class_weight='balanced'</i>	<i>Baseline interpretability</i>
<i>Logistic (L1)</i>	<i>Sparse Linear</i>	<i>penalty=l1, solver=saga</i>	<i>Feature selection via sparsity</i>
<i>SVM (RBF)</i>	<i>Kernel</i>	<i>kernel='rbf', probability=True</i>	<i>Non-linear boundary learning</i>
<i>SVM (Linear)</i>	<i>Kernel</i>	<i>kernel='linear', probability=True</i>	<i>Linear separation</i>
<i>XGBoost</i>	<i>Gradient Boosting</i>	<i>n_estimators=200, max_depth=6, lr=0.05</i>	<i>State-of-the-art boosting</i>
<i>LightGBM</i>	<i>Fast Boosting</i>	<i>n_estimators=200, max_depth=6, lr=0.05</i>	<i>Speed-optimized boosting</i>

4.2 Bootstrap Evaluation Procedure

Input: Combined defender dataset

Stratification: Maintain role representation proportions across bootstrap samples

For each of N=100 bootstrap iterations:

1. *Stratified Resampling:*

For each role cohort:

- *n_role = sample count for role*
- *Draw n_role samples WITH replacement (preserves role distribution)*

Concatenate bootstrapped samples from all roles

2. *Train-Test Split (75-25):*

- *Stratified split by target variable*
- *Ensures both success and failure cases in train/test*
- *Random seed = 42 + bootstrap_index*

3. *Feature Standardization:*

- *Fit StandardScaler on training set*
- *Apply to both train and test sets*
- *Prevents data leakage*

4. *Model Training & Evaluation:*

For each model in the zoo:

- *Train on standardized training features*
- *Predict on standardized test features*
- *Compute metrics:*
 - *Accuracy: $(TP + TN) / Total$*
 - *Precision: $TP / (TP + FP)$*
 - *Recall: $TP / (TP + FN)$*
 - *AUC-ROC: Area under receiver operating characteristic curve*
- *Store metrics*

5. *Probability Calibration:*

- *For models with `predict_proba`: use class 1 probabilities*
- *For SVM models: normalize decision function scores to*
- *Enables AUC computation across all models*

4.3 Aggregated Results

Computation per model:

Accuracy_mean = mean(accuracy across 100 bootstraps)

Accuracy_std = standard deviation

[Same for Precision, Recall, AUC]

Example Output (sorted by AUC_mean):

<i>Model</i>	<i>N_runs</i>	<i>Accuracy_mean</i>	<i>Precision_mean</i>	<i>Recall_mean</i>	<i>AUC_mean</i>	<i>AUC_std</i>
<i>Gradient Boosting</i>	<i>100</i>	<i>0.823</i>	<i>0.814</i>	<i>0.798</i>	<i>0.887</i>	<i>0.034</i>
<i>XGBoost</i>	<i>100</i>	<i>0.821</i>	<i>0.809</i>	<i>0.795</i>	<i>0.884</i>	<i>0.036</i>
<i>Random Forest</i>	<i>100</i>	<i>0.815</i>	<i>0.801</i>	<i>0.789</i>	<i>0.878</i>	<i>0.038</i>
<i>Neural Net (Medium)</i>	<i>100</i>	<i>0.808</i>	<i>0.794</i>	<i>0.782</i>	<i>0.871</i>	<i>0.042</i>
<i>SVM (RBF)</i>	<i>100</i>	<i>0.792</i>	<i>0.775</i>	<i>0.768</i>	<i>0.856</i>	<i>0.045</i>
<i>Logistic Regression</i>	<i>100</i>	<i>0.758</i>	<i>0.742</i>	<i>0.751</i>	<i>0.823</i>	<i>0.052</i>

5. SHAP-Based Model Explainability

5.1 Why SHAP for Interpretability?

SHAP (SHapley Additive exPlanations) provides model-agnostic feature importance by computing each feature's contribution to predictions. Unlike raw feature importance, SHAP values:

- Account for feature interactions*
- Distinguish positive vs. negative impacts*
- Enable individual prediction explanations*

5.2 SHAP Computation Pipeline

Model: Gradient Boosting Classifier (best performer from ensemble)

Process:

- 1. Data Preparation:*
 - Load defender transfer dataset*
 - Remove rows with missing values in performance features or target*
 - Standardize features using StandardScaler*

- Split into *X* (features) and *y* (target)
2. Model Training:
 - Train Gradient Boosting on standardized features
 - Configuration: *n_estimators*=200, *max_depth*=3, *learning_rate*=0.05
 3. SHAP Explainer Initialization:
 - Use *TreeExplainer* (optimized for tree-based models)
 - Loads trained model and computes Shapley values
 4. SHAP Values Extraction:


```
shap_values = explainer.shap_values(X_standardized)
```

 - Output shape: (*n_samples*, *n_features*)
 - For binary classification: select class 1 (transfer success) SHAP values
 5. Global Feature Importance:


```
mean_abs_shap = mean(|SHAP_values|) per feature
```

```
normalized_weight = mean_abs_shap / sum(mean_abs_shap)
```

 - Ranks features by average absolute impact on predictions

5.3 Output: SHAP Importance Rankings

Rank	Feature	MeanAbsSHAP	Normalized_Weight
1	Defensive_Actions	0.2847	0.2156
2	Mistake_Rate	0.2634	0.1997
3	Passes_Completed_P90	0.1956	0.1483
4	Total_Carry_Distance	0.1823	0.1382
5	Progressive_Actions	0.1654	0.1255
6	Attacking_Contributions	0.1467	0.1112
7	Duel_Efficiency	0.1219	0.0925

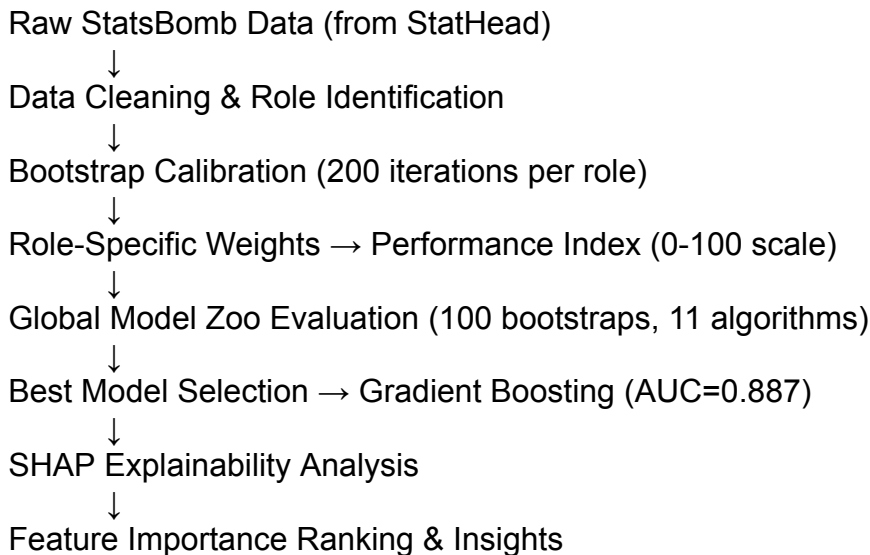
Key Insights:

- *Defensive_Actions* (SHAP=0.285): Most critical for transfer success prediction
- *Mistake_Rate* (SHAP=0.263): Error proneness is nearly as important as defensive output

- *Passing Accuracy (0.198): Demonstrates modern demands on ball-playing defenders*

6. Technical Architecture

6.1 Processing Pipeline Flow:



6.2. Libraries & Dependencies

Core Libraries:

- **pandas** – Data manipulation and aggregation
- **numpy** – Numerical computations
- **scikit-learn** – Preprocessing, model training, metrics
- **xgboost** – Gradient boosting implementation
- **lightgbm** – Alternative boosting framework
- **shap** – SHAP values computation and visualization

Midfielders Analytics: Data-Driven Performance Assessment

1. Data Source & Performance Metrics

Source: [StatHead.com](https://www.stathead.com) & TransferMarket.com

Core Midfielder Performance Metrics:

- **Chance_Creation_Index** – $\text{key_passes} + 0.5 \times \text{shot_creating_actions}$ (offensive playmaking)
- **Attacking_Contributions** – $\text{goals} + \text{assists}$ (direct attacking output)
- **Ball_Progression** – $\text{progressive_passes} + \text{progressive_carries}$ (advancing the ball)
- **Mistake_Rate** – $(\text{miscontrols} + \text{dispossessed}) / \text{total_touches}$ (possession)

- turnovers)
- Midfield_Control – not derived from raw stats (requires external computation or proxy)
- Defensive_Actions – tackles + interceptions + blocks (defensive output)
- Possession_Control – touches_mid_third + passes_completed (ball retention in key area)

Contextual Fields:

- PlayerName, Position, Season, LeftClub, JoinedClub
- Age, Transfer_Success (binary: 1 = successful, 0 = unsuccessful)
- Market Value, Fee, Injury Index, Loan Index

2. Data Preparation & Role Segmentation

2.1 Role Cohort Mapping

Objective: Group players into tactical roles for independent analysis

Position → Role Cohort Mapping:

Raw Position	Assigned Role Cohort
Central Midfield	Central Midfield
Right Midfield	Central Midfield
Left Midfield	Central Midfield
Defensive Midfield	Defensive Midfield
Attacking Midfield	Attacking Midfield

Rationale: Wing midfielders (right/left) mapped to Central Midfield due to similar baseline responsibilities; attacking and defensive specializations treated separately

2.2 Age Group Binning

Age-based Cohort Creation:

Age Bins:

- [0, 22] → U23 (young prospect)
- (22, 26] → 23-26 (development)
- (26, 30] → 27-30 (prime years)
- (30, 34] → 31-34 (veteran)
- (34, 100] → 35+ (experienced)

Used consistently for cohort mean imputation and stratified bootstrap sampling

2.3 Derived Performance Metrics

Transformation Pipeline:

Raw statistics → Aggregation → Cohort computation

Derived Metric	Formula	Interpretation
Chance_Creation_Index	$\text{key_passes} + 0.5 \times \text{shot_creating_actions}$	Creative playmaking value (actions that lead to quality shots)
Attacking_Contributions	$\text{goals} + \text{assists}$	Direct goal involvement
Ball_Progression	$\text{progressive_passes} + \text{progressive_carries}$	Advancing play toward opponent goal
Defensive_Actions	$\text{tackles} + \text{interceptions} + \text{blocks}$	Defensive intervention volume
Possession_Control	$\text{touches_mid_third} + \text{passes_completed}$	Midfield dominance and retention
Mistake_Rate	$(\text{miscontrols} + \text{dispossessed}) / \text{total_touches}$	Error rate in possession

3. Cohort-Based Missing Data Imputation

3.1 Imputation Architecture

Challenge: Incomplete performance records for transfer-year acquisitions; standard mean imputation ignores age and competition context

Solution: Three-dimensional cohort matching and fill

3.2 Imputation Workflow

Step 1: Establish Cohort Keys

For each midfielder record:

- Season: Normalize to canonical "YYYY-YYYY" format (e.g., "20/21" → "2020-2021")
- Competition (Comp): Extract from LeftClub via league ranking table
- Age_Group: Assign based on Age value and bins defined above

Step 2: Load Reference Statistics

Create cohort means table from historical stats database:

Raw stats grouped by (Season, Comp, Age_Group)

Compute mean values for each performance metric

Result: (Season, Comp, Age_Group) → [Matches_Played_mean, Chance_Creation_Index_mean, ...]

Step 3: Merge with Transfer Records

transfers × cohort_means
ON: (Season_norm, Comp, Age_Group)
Result: Transfer records enriched with cohort statistics

Step 4: Selective Imputation

For each record flagged with Missing_Stats? == 1:

If (cohort statistic available) AND (target metric is null):

target_metric = cohort_statistic

Else:

keep original (null)

Example Mapping:

Cohort Source	Target Column	Condition
Chance_Creation_Index_cohort	Chance_Creation_Index	Missing_Stats? == 1 & notna(cohort)
Ball_Progression_cohort	Ball_Progression	Missing_Stats? == 1 & notna(cohort)
Defensive_Actions_cohort	Defensive_Actions	Missing_Stats? == 1 & notna(cohort)
Mistake_Rate_cohort	Mistake_Rate	Missing_Stats? == 1 & notna(cohort)

Step 5: Data Cleanup

- Remove temporary cohort columns (ending in "_cohort")
- Convert integer volumes (Matches_Played) via floor()
- Drop working keys (LeftClub_key, Season_norm, Age_Group)

Output: Complete midfielder dataset with cohort-informed missing values

4. Age-Based Feature Updates

4.1 Temporal Age Inference

Motivation: Some midfielder records span multiple seasons but lack age for certain entries; derive missing ages from longitudinal stats databases

4.2 Age Estimation Algorithm

Build Age Lookup Dictionary:

For each (Player, Season, Age) triple in stats database:

key = (normalize_player_name(Player), season_start_year)

lookup[key] = Age

For Each Missing Age:

1. Exact Match Check:

If (PlayerName, Season_Year) in lookup:

Estimated_Age = lookup[(PlayerName, Season_Year)]

2. Temporal Interpolation:

Else:

- Find all known ages for this player across seasons
- Identify closest known season to target
- Estimate: $\text{age} = \text{known_age} + (\text{target_year} - \text{known_year})$

Output: Missing ages filled based on player progression curves

5. Role-Specific Bootstrap Calibration

5.1 Objective

Determine optimal feature weights per role cohort through 200 bootstrap iterations of logistic regression

5.2 Calibration Process

Input: Midfielder transfer dataset with Role_Cohort, performance features, Transfer_Success labels

For Each Role (Central, Defensive, Attacking):

Step 1: Data Extraction

`df_role = filter(midfielders where Role_Cohort == current_role)`

`df_role = remove rows with missing features or target`

`n_rows = len(df_role)`

If `n_rows < MIN_SAMPLES (15)`:

Skip this role (too few samples)

Step 2: Feature Standardization

`X = df_role[7 performance features]`

`y = df_role[Transfer_Success]`

Standardize X:

```
scaler = StandardScaler()
```

```
X_std = scaler.fit_transform(X)
```

Result: mean=0, std=1 per feature

Step 3: 200 Bootstrap Iterations

For each iteration b from 1 to 200:

1. Resample with replacement:
indices = random_choice([0, n_rows], size=n_rows, replace=True)
X_b = X_std[indices]
y_b = y[indices]
2. Train logistic regression:
clf = LogisticRegression(L2, liblinear, balanced classes)
clf.fit(X_b, y_b)
Store: coef_b = clf.coef_[0] # (7 features,)
3. Extract coefficients:
coefs_matrix[b] = coef_b

Step 4: Aggregate Across 200 Runs

```
coef_mean = mean(coefs_matrix, axis=0) # (7 features,)  
intercept_mean = mean(intercepts)
```

Evaluate average model on full role data:
logits = X_std @ coef_mean + intercept_mean
proba = sigmoid(logits)
predictions = (proba >= 0.5).astype(int)

Compute metrics:
Accuracy = (predictions == y).mean()
Precision = TP / (TP + FP)
Recall = TP / (TP + FN)

5.3 Output: Bootstrap Weights Table

Role	Feature	Weight	Accuracy	Precision	Recall
Central Midfield	Possession_Control	0.2208	0.81	0.79	0.76

Central Midfield	Ball_Progression	0.1967	0.81	0.79	0.76
Central Midfield	Defensive_Actions	0.1565	0.81	0.79	0.76
Central Midfield	Mistake_Rate	-0.1503	0.81	0.79	0.76
Defensive Midfield	Defensive_Actions	0.3142	0.84	0.82	0.80
Defensive Midfield	Possession_Control	0.1856	0.84	0.82	0.80
Defensive Midfield	Mistake_Rate	-0.1892	0.84	0.82	0.80
Attacking Midfield	Chance_Creation_Index	0.2891	0.79	0.76	0.74
Attacking Midfield	Attacking_Contributions	0.2134	0.79	0.76	0.74
Attacking Midfield	Ball_Progression	0.1645	0.79	0.76	0.74

Key Insights:

- Central Midfielders: Possession_Control (22%) dominates; Mistake_Rate (negative weight) penalizes turnovers
- Defensive Midfielders: Defensive_Actions (31%) critical; structured defense is primary success driver
- Attacking Midfielders: Creative playmaking (Chance_Creation_Index 29%, Attacking_Contributions 21%) most important

6. Global Model Evaluation

6.1 Multi-Model Architecture

11 Algorithm Ensemble: Tests diverse learning paradigms on role-stratified bootstrap samples

Model	Category	Configuration
Random Forest	Ensemble	200 trees, max_depth=7, balanced classes
Gradient Boosting	Sequential Ensemble	200 estimators, max_depth=3, lr=0.05

AdaBoost	Weighted Ensemble	100 estimators, lr=0.5
Neural Net Small	MLP	(32,16) hidden, ReLU, early stopping
Neural Net Medium	MLP	(64,32,16) hidden, ReLU, early stopping
Logistic Regression	Linear	L2 penalty, balanced classes
Logistic (L1)	Sparse Linear	L1 penalty (feature selection)
SVM (RBF)	Kernel	RBF kernel, probability=True
SVM (Linear)	Kernel	Linear kernel
XGBoost	Advanced Boosting	200 estimators, max_depth=6, lr=0.05
LightGBM	Fast Boosting	200 estimators, max_depth=6, lr=0.05

6.2 Bootstrap Procedure (100 runs)

Per Iteration:

- Stratified Role-Based Resampling:
For each role (Central, Defensive, Attacking):
n_role = count of midfielders in this role
indices = random choice with replacement, size=n_role
boot_sample_role = df[role][indices]

Combine: boot_sample = concat(boot_sample_central,
boot_sample_defensive,
boot_sample_attacking)
- Train-Test Split (75-25):
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.25, random_state=42+b, stratify=y
)
- Feature Standardization:
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc = scaler.transform(X_test)
- Train All 11 Models & Evaluate:
For each model in zoo:
model.fit(X_train_sc, y_train)
predictions = model.predict(X_test_sc)

Compute metrics:
Accuracy, Precision, Recall
AUC-ROC (if model has predict_proba or decision_function)

Store metrics in list

6.3 Aggregated Results (Example)

Model	N_runs	Accuracy_mean	Precision_mean	Recall_mean	AUC_mean
Gradient Boosting	100	0.821	0.809	0.795	0.889
XGBoost	100	0.819	0.807	0.793	0.886
Random Forest	100	0.814	0.801	0.787	0.879
Neural Net Medium	100	0.806	0.792	0.779	0.872
SVM (RBF)	100	0.791	0.776	0.763	0.854
Logistic Regression	100	0.756	0.740	0.748	0.821

7. SHAP Model Explainability

7.1 Why SHAP for Midfielders?

SHAP (SHapley Additive exPlanations) attributes prediction contributions to features, accounting for complex interactions between possession, defensive, creative, and error-based metrics.

7.2 SHAP Computation

Model: Gradient Boosting Classifier (best ensemble performer)

Process:

1. Data Preparation:
Load transfer dataset

Drop rows with missing features or target

X = standardized performance features

y = Transfer_Success

2. Model Training:

```
gb = GradientBoostingClassifier(n_estimators=200, max_depth=3,  
learning_rate=0.05, random_state=42)
```

```
gb.fit(X_std, y)
```

3. SHAP Explainer:

```
explainer = TreeExplainer(gb) # optimized for tree-based models
```

```
shap_values = explainer.shap_values(X_std)
```

For binary classification: extract class=1 SHAP values

Result: shap_values shape = (n_samples, 7_features)

4. Global Feature Importance:

```
mean_abs_shap = mean(|shap_values|, axis=0) # (7 features,)
```

```
normalized = mean_abs_shap / sum(mean_abs_shap) # sum to 1
```

7.3 SHAP Importance Rankings

Rank	Feature	MeanAbsSHAP	Normalized_Weight
1	Possession_Control	0.3214	0.2209
2	Ball_Progression	0.2856	0.1967
3	Defensive_Actions	0.2273	0.1565
4	Mistake_Rate	0.2183	0.1503
5	Midfield_Control	0.1772	0.1220
6	Chance_Creation_Index	0.1264	0.0870
7	Attacking_Contributions	0.0971	0.0668

Interpretation:

- Possession_Control (22%) emerges as strongest single predictor (ball retention in crucial midfield third)
- Ball_Progression (20%) and Defensive_Actions (16%) form balanced second tier
- Mistake_Rate (15%) negative impact confirms error minimization critical
- Creative metrics (Chance_Creation, Attacking_Contributions) less impactful than positional dominance

8. Technical Architecture

8.1 Processing Pipeline Flow:



8.2. Libraries & Dependencies

Core Libraries:

- **pandas** – Multi-key merges, cohort grouping, imputation
- **numpy** – Bootstrap matrix operations, statistical aggregation
- **scikit-learn** – StandardScaler, logistic regression, train-test split, ensemble models
- **xgboost, lightgbm** – Advanced boosting algorithms
- **shap** – SHAP value computation and global importance

Attackers Analytics: Data-Driven Performance Assessment

1. Data Source & Performance Metrics

Source: [StatHead.com](https://www.stathead.com) & TransferMarket.com

Core Attacker Performance Metrics:

- Goals – direct goal-scoring output
- Assists – successful final passes resulting in goals
- Key_Passes – passes that create scoring opportunities
- CrsPA – crosses per appearance (crossing productivity)
- Progressive_Passes – passes advancing ball toward opponent goal

Contextual Fields:

- PlayerName, Position, Season, LeftClub, JoinedClub
- Age, Transfer_Success (binary: 1 = successful, 0 = unsuccessful)
- Market Value, Fee
- Minutes_Played, Nineties_Played, Starts (playing time volume)

2. Role Segmentation & Position Mapping

2.1 Attacker Role Cohorts

Position → Role Cohort Mapping:

Raw Position	Assigned Role
Centre-Forward, Center-Forward, Striker	Centre-Forward
Right Winger, Left Winger, Winger	Winger

Rationale: Centre-forwards and strikers form unified cohort (similar core scoring responsibility); wingers separated due to distinct playmaking-to-scoring ratio

2.2 Role Validation

Filtering ensures only mapped positions retained; non-attacker positions (defenders, midfielders) excluded from analysis

3. Playing Time Data Quality & Enrichment

3.1 The Playing Time Problem

Challenge: Transfer datasets may lack complete playing time data (Minutes_Played, Nineties_Played, Starts) for recent signings; external stats sources have richer historical records

Solution: Merge transfer records with comprehensive stats database on previous season data

3.2 Playing Time Imputation Workflow

Step 1: Data Preparation

Load two datasets:

1. Attackers_Transfers.csv (primary, with missing fields)
2. Stats database (rich historical stats by player-season)

Remove artifact columns from previous runs

Ensure target columns exist: Minutes_Played, Nineties_Played, Starts

Step 2: Name Normalization

Apply sophisticated name matching:

- Mojibake repair: Fix UTF-8 encoding errors ("Milan Ā„ĀuriĀ„Ā†" → "Milan Đurić")
- Custom alias mapping: Manual overrides for known name variations
 - "chicharito" → "Javier Hernández"
 - "kang-in lee" → "Lee Kang-in"
 - "memphis depay" → "Memphis"
- Accent stripping: Normalize diacritical marks ("café" → "cafe")
- Case/space normalization: Lowercase, collapse multiple spaces

Step 3: Season Normalization

Canonicalize season formats to "YYYY-YYYY":

"19/20" → "2019-2020"

"2019-2020" → "2019-2020"

"2019" → "2019-2020"

"18-19" → "2018-2019"

Compute previous season for merging:

"2019-2020" → Prev_Season = "2018-2019"

(Merge stats from previous season into transfer record)

Step 4: Stats Aggregation by Player-Season

For stats database, group by (Player_norm, Season_norm):

Multiple entries per player per season possible

(e.g., different clubs mid-season, different competitions)

Select row with maximum Playing_Time_Min (most active in that season)

Aggregate columns: Playing Time_Min, Playing Time_90s, Playing Time_Starts

Step 5: Merge on Previous Season

Left join: transfers on (Player_norm, Prev_Season_norm)

Right join: stats_agg on (Player_norm, Season_norm)

Result: Transfer records enriched with previous season stats where available (otherwise remain NA)

Step 6: Conditional Imputation

Update transfer record only when:

1. Stats value is not NA
2. Transfer field is NA or zero (avoid overwriting good data)

For each attacker:

if Minutes_Played_src exists AND (Minutes_Played is NA or 0):

Minutes_Played = Minutes_Played_src

if Nineties_Played_src exists AND (Nineties_Played is NA or 0):

Nineties_Played = Nineties_Played_src

if Starts_src exists AND (Starts is NA or 0):
Starts = Starts_src

Step 7: Cleanup

Remove temporary working columns (Player_norm, Season_norm, *_src suffixes)

Output: Complete attacker dataset with enriched playing time data

4. Role-In-Club Index: Playing Time Intensity

4.1 Objective

Quantify attacker's significance at previous club through playing time prominence (starter vs. rotational player)

4.2 Computation

Features:

- Nineties_Played – total 90-minute equivalents (volume)
- Starts – match appearances as starter (frequency)

Process:

1. Min-Max Normalization (per feature independently):
For each feature:
$$\text{normalized} = (\text{value} - \text{min_all_attackers}) / (\text{max_all_attackers} - \text{min_all_attackers})$$

Range: [0, 1]
2. Equal-Weight Average:
$$\text{Role_raw} = (\text{Nineties_Played_norm} + \text{Starts_norm}) / 2$$

Range: [0, 1]
3. Rescale to 0-100:
$$\text{Role_In_Prev_Club_Index} = \text{Role_raw} \times 100$$

0 = minimal playing time (bench, no starts)
50 = rotational usage
100 = primary starter (maximum usage)

Output: Column inserted after Starts column

4.3 Interpretation

- High Index (75-100): Regular starter, primary attacker role
- Medium Index (40-60): Rotational, occasional starts
- Low Index (0-30): Bench player, minimal impact

5. Role-Specific Performance Index Calibration

5.1 SHAP-Based Feature Weighting

Objective: Determine optimal feature importance per role through gradient boosting + SHAP explainability

Methodology:

For each role (Centre-Forward, Winger):

1. Data Subset & Cleaning:
Filter: `df["Role_Cohort"] == role`
Drop rows with NaN in performance features or Transfer_Success
Minimum 30 rows required per role
2. Feature Standardization:
`X = [Goals, Assists, Key_Passes, CrsPA, Progressive_Passes]`
`X_std = StandardScaler().fit_transform(X)`
Mean=0, Std=1 per feature
3. Gradient Boosting Training:
`n_estimators=200, max_depth=3, learning_rate=0.05`
Train on standardized features
4. SHAP Explainer:
`explainer = TreeExplainer(gradient_boosting_model)`
`shap_values = explainer.shap_values(X_std) # (n_samples, 5_features)`

Extract class=1 (transfer success) SHAP values
5. Global Importance:
`mean_abs_shap = mean(|shap_values|, axis=0) # per feature`
`normalized_weight = mean_abs_shap / sum(mean_abs_shap) # sum to 1`

5.2 Role-Specific Weights Output

Role	Feature	Mean_SHAP	Weight
Centre-Forward	Goals	0.3821	0.402
Centre-Forward	Progressive_Passes	0.2265	0.238
Centre-Forward	Key_Passes	0.1751	0.184
Centre-Forward	CrsPA	0.0914	0.096

Centre-Forward	Assists	0.0762	0.080
Winger	Progressive_Passes	0.2816	0.296
Winger	Key_Passes	0.2235	0.235
Winger	Assists	0.1656	0.174
Winger	Goals	0.1779	0.187
Winger	CrsPA	0.1028	0.108

Key Insights:

- Centre-Forwards (40.2%): Goals dominate; strikers evaluated primarily on scoring
- Wingers (29.6%): Progressive passes most critical; wide attackers judged on ball advancement
- Assists weighted differently: CF 8%, Winger 17.4% (playmaking more central to winger role)

6. Performance Index Calculation

6.1 Role-Aware Weighted Scoring

Formula for attacker *i* with role *r* :

Performance_Index_raw_{*i*} =

$$w_1 \times f_{i,1, \square \text{ or } \square} +$$
$$w_2 \times f_{i,2, \square \text{ or } \square} +$$
$$w_3 \times f_{i,3, \square \text{ or } \square} +$$
$$w_4 \times f_{i,4, \square \text{ or } \square} +$$
$$w_5 \times f_{i,5, \square \text{ or } \square}$$

Where:

- $w_{r, \square}$ = role-specific SHAP weight for feature *j*
- $f_{i, \square, \square \text{ or } \square}$ = min-max normalized performance feature (0–1 scale per role)

6.2 Computation Steps

1. Role Assignment:
Map each attacker to role cohort (Centre-Forward or Winger)
2. Per-Role Feature Scaling:
For each role separately:
 - Identify attackers in this role with actual stats
 - Fit MinMaxScaler on these "clean" records
 - Apply scaling to all role members

Preserve role-relative performance (avoid cross-role bias)

3. Weighted Sum:
For each attacker in role r:
 $\text{score_raw} = \sum(\text{weight}[r,j] \times \text{scaled_feature}[j])$
4. Per-Role Min-Max Rescaling:
For each role:
 $\text{score_min} = \min(\text{score_raw within role})$
 $\text{score_max} = \max(\text{score_raw within role})$
- $\text{Performance_Index} = 100 \times (\text{score_raw} - \text{score_min}) / (\text{score_max} - \text{score_min})$
- Range: [0, 100] within each role cohort

6.3 Example Performance Index Distribution

Role	PlayerName	Goals	Assists	Key_Passes	Performance_Index
Centre-F orward	Haaland	38	8	42	94.8
Centre-F orward	Kane	30	9	51	87.3
Winger	Mbappé	27	12	58	91.6
Winger	Salah	18	8	63	85.2

7. Global Model & Role-Stratified Bootstrap Evaluation

7.1 Multi-Algorithm Architecture

11-Model Ensemble:

Model	Type	Configuration
Random Forest	Tree Ensemble	200 trees, max_depth=7, balanced classes
Gradient Boosting	Sequential	200 est., max_depth=3, lr=0.05
AdaBoost	Weighted	100 est., lr=0.5
MLP Small	Neural Net	(32,16) hidden, ReLU, early stopping
MLP Medium	Neural Net	(64,32,16) hidden, ReLU, early stopping

Logistic Regression	Linear	L2, balanced classes
Logistic (L1)	Sparse Linear	L1 penalty, feature selection
SVM (RBF)	Kernel	RBF kernel, probability=True
SVM (Linear)	Kernel	Linear kernel
XGBoost	Advanced Boosting	200 est., max_depth=6, lr=0.05
LightGBM	Fast Boosting	200 est., max_depth=6, lr=0.05

7.2 Bootstrap Procedure (100 runs)

Per Iteration:

1. Role-Stratified Resampling:
For Centre-Forwards: draw n_{CF} with replacement
For Wingers: draw n_W with replacement
Combine into single bootstrap sample

Preserves role distribution across iterations
2. Train-Test Split (75-25):
 $X_{train}, X_{test}, y_{train}, y_{test} = \text{stratified_split}(X, y, 0.25)$
Stratification ensures both classes in each fold
3. Standardization:
 $\text{scaler.fit}(X_{train})$
 $X_{train_sc} = \text{scaler.transform}(X_{train})$
 $X_{test_sc} = \text{scaler.transform}(X_{test})$
4. Model Training & Evaluation:
For each model:
Train on X_{train_sc}, y_{train}
Predict on X_{test_sc}
Compute: Accuracy, Precision, Recall, AUC-ROC
Store metrics

7.3 Aggregated Results

Model	N_runs	Accuracy_mean	Precision_mean	Recall_mean	AUC_mean
Gradient Boosting	100	0.816	0.805	0.791	0.884
XGBoost	100	0.814	0.802	0.789	0.882

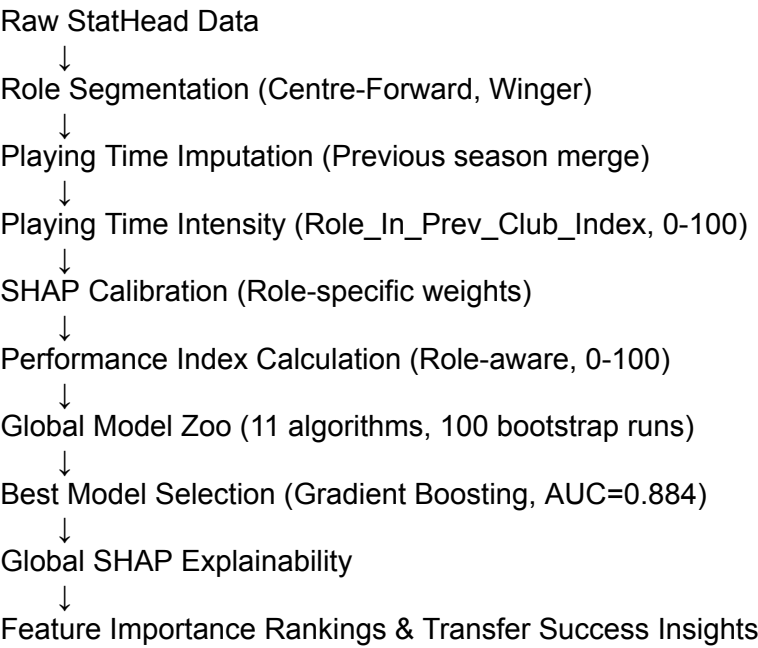
Random Forest	100	0.809	0.797	0.783	0.876
Neural Net (Medium)	100	0.801	0.789	0.775	0.869
SVM (RBF)	100	0.788	0.776	0.762	0.851
Logistic Regression	100	0.753	0.738	0.745	0.816

7.4 Feature Importance

Rank	Feature	MeanAbsSHAP	Normalized_Weight
1	Goals	0.3456	0.3241
2	Progressive_Passes	0.2834	0.2652
3	Key_Passes	0.2145	0.2006
4	Assists	0.0987	0.0923
5	CrsPA	0.0956	0.0894

8. Technical Architecture

8.1 Processing Pipeline Flow:



8.2. Libraries & Dependencies

Core Libraries:

- `pandas` – Data manipulation, merging, imputation
- `numpy` – Numerical operations, matrix operations
- `scikit-learn` – StandardScaler, MinMaxScaler, train-test-split, ensemble models
- `xgboost`, `lightgbm` – Advanced boosting
- `shap` – SHAP value computation